

Increasing Solution Density by Dominated Relaxation

Steven Prestwich

Cork Constraint Computation Centre
Department of Computer Science
University College Cork, Ireland
s.prestwich@cs.ucc.ie

Abstract. The solution density of a constraint problem can have a significant effect on local search performance. This paper describes a reformulation technique for artificially increasing solution density: relaxing the model in such a way that relaxed solutions are dominated by real solutions, and can be transformed into them. On an open stack minimization problem this technique can exponentially increase solution density, boosting local search performance.

1 Introduction

The *solution density* of a constraint model may be defined as the number of solutions divided by the number of total variable assignments. The idea that higher solution density should make a problem easier to solve is natural, particularly for a local search algorithm [5, 12, 14, 17, 20, 22], but little has been done to exploit this conjecture in practice. This paper describes a reformulation technique that uses relaxation with dominance to increase solution density. We relax a model in such a way that any relaxed solution is dominated by at least one real solution, and can be transformed into it. This increases the solution density of the model and boosts local search performance (we shall ignore its effect on back-track search, but discuss this issue in Section 4). We shall call this a *dominated relaxed model* and the new solutions *pseudo-solutions*.

We illustrate the idea on the *minimization of open stacks problem* (MOSP) used in the First Constraint Modelling Challenge,¹ and studied in several papers including [9, 21]. A manufacturer has a number of orders from customers to satisfy. Each order is for a number of different products, and only one product can be made at a time. Once a customer's order is started a stack is created for that customer. When all the products that a customer requires have been made the order is sent to the customer, so that the stack is closed. Because of limited space in the production area, the number of stacks that are simultaneously open should be minimized.

¹ <http://www.dcs.st-and.ac.uk/~ipg/challenge/>

1.1 A matrix representation

MOSP can be modelled as a matrix M of binary variables, in which the columns correspond to the products required by the customers, and the rows to the customers' orders. Matrix entry $M_{ij} = 1$ if and only if customer i has ordered some quantity of product j (the quantity ordered is irrelevant). Any 0 with a 1 in a column to its left, and another 1 in a column to its right, is counted as a 1. A score is assigned to each column: its number of 1s (including 0s counted as 1s). The score of the matrix is the maximum of its column scores. The problem is to permute the columns of the matrix to minimise its score. An example is shown in Figure 1. The zeros in **bold** have 1s to their left and right and are counted as 1s. The matrix score is 3, which is the maximum column score. The column permutation shown is optimal.

$$\begin{array}{cccccc}
 0 & 1 & \mathbf{0} & \mathbf{0} & 1 & \\
 0 & 0 & 1 & 1 & 0 & \\
 1 & 0 & 0 & 0 & 0 & \\
 0 & 0 & 1 & 1 & 1 & \\
 \hline
 0 & 1 & 0 & 0 & 0 & \\
 \hline
 1 & 2 & 3 & 3 & 2 &
 \end{array}$$

Fig. 1. A MOSP instance with column scores

1.2 An integer model

The matrix representation can be modelled as an integer program. Suppose the matrix has R rows and C columns. Assume that each customer orders at least one product (if not then that customer can be removed from the problem), so that every row has an open order of length at least 1. In the following, i is an integer with range $1 \dots R$ and j, k are integers with range $1 \dots C$. Define binary variables p_{jk} such that $p_{jk} = 1$ if and only if product j is placed in column k . Each product must be placed in exactly one column and each column must receive exactly one product:

$$\sum_k p_{jk} = 1 \quad \sum_j p_{jk} = 1 \quad (1)$$

To model the idea of 1s influencing 0s to their left and right, define variables l_{ij} and r_{ij} such that $l_{ik} = 1$ if and only if M_{ik} has a 1 to its left, and $r_{ik} = 1$ if and only if M_{ik} has a 1 to its right. Add constraints

$$p_{jk} \leq l_{ik} \quad p_{jk} \leq r_{ik} \quad (2)$$

for i, j such that $M_{ij} = 1$, and

$$l_{ik} \leq l_{i, k+1} \quad r_{i, k+1} \leq r_{ik} \quad (3)$$

Also define variables o_{ik} such that $o_{ik} = 1$ if and only if there is an open order on row i at column k :

$$l_{ik} + r_{ik} \leq 1 + o_{ik} \quad (4)$$

The objective is to find consistent values for the p, l, r, o variables while minimising

$$\max_k \sum_i o_{ik}$$

This optimisation problem can be solved as a series of constraint satisfaction problems (CSPs) with additional linear constraints:

$$\sum_i o_{ik} \leq \Omega \quad (5)$$

where Ω is an integer variable with range $0 \dots R$. The objective is to minimise Ω . Starting with $\Omega = R$ a feasible solution is found; R is then decremented and the search resumed with the same variable assignments (to exploit any solution clustering); and so on until timeout occurs or a desired value is reached.

1.3 Creating pseudo-solutions

We would like to remove or weaken some constraints in order to create new pseudo-solutions, thus increasing the solution density of the problem. But we cannot simply remove arbitrary constraints, because solutions to the resulting problem might not be MOSP solutions, or they might be solutions with larger scores. We must do it in such a way that any pseudo-solution can be transformed into a solution of equal or lower score: a *dominating* solution. Suppose we weaken constraints (1) to

$$\sum_k p_{jk} \geq 1 \quad (6)$$

Now each product may be placed in more than one position in the sequence, and some sequence positions might receive no products. At first sight this appears useless because solutions might not be permutations. However, it can be seen as a model for a generalised problem: find a *sequence of sets of products* such that each product appears in at least one set, and the orders are open in set k if there is a product required in an order in another set i that appears in a set before or after set k , or in k itself. Such a sequence is a pseudo-solution for MOSP.

For example, consider the 5×5 MOSP given by the matrix in Figure 2(i). A solution with stack usage 3 is the product sequence (4,5,1,2,3) giving the permutation in Figure 2(ii). A pseudo-solution

$$(\{4\}, \{4\}, \{4, 5\}, \{\}, \{1, 2, 3\})$$

that is dominated by this permutation is shown in Figure 2(iii), so there are at least two representations of the permutation. In fact there are several others: for example the empty set can occur anywhere in the sequence, and two of the three occurrences of product 4 can be removed. It can be verified that all these pseudo-solutions are optimal because they have score 3.

<table> <tr><td>1</td><td>2</td><td>3</td><td>4</td><td>5</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td>1</td></tr> </table>	1	2	3	4	5	0	0	1	0	1	1	1	0	0	0	0	0	0	1	0	1	1	1	0	0	0	0	0	0	1	<table> <tr><td>4</td><td>5</td><td>1</td><td>2</td><td>3</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>0</td><td>1</td><td>1</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td><td>0</td><td>0</td></tr> </table>	4	5	1	2	3	0	1	0	0	1	0	0	1	1	0	1	0	0	0	0	0	0	1	1	1	0	1	0	0	0	<table> <tr><td>4</td><td>4</td><td>4</td><td>5</td><td></td><td>1</td><td>2</td><td>3</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>1</td><td></td><td></td><td></td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td></td><td></td><td></td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td><td>0</td><td></td><td></td><td></td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>0</td><td></td><td></td><td></td><td>0</td></tr> <tr><td>0</td><td>0</td><td>0</td><td>1</td><td></td><td></td><td></td><td>0</td></tr> </table>	4	4	4	5		1	2	3	0	0	0	1				0	0	0	0	0				0	1	1	1	0				0	0	0	0	0				0	0	0	0	1				0
1	2	3	4	5																																																																																																										
0	0	1	0	1																																																																																																										
1	1	0	0	0																																																																																																										
0	0	0	1	0																																																																																																										
1	1	1	0	0																																																																																																										
0	0	0	0	1																																																																																																										
4	5	1	2	3																																																																																																										
0	1	0	0	1																																																																																																										
0	0	1	1	0																																																																																																										
1	0	0	0	0																																																																																																										
0	0	1	1	1																																																																																																										
0	1	0	0	0																																																																																																										
4	4	4	5		1	2	3																																																																																																							
0	0	0	1				0																																																																																																							
0	0	0	0				0																																																																																																							
1	1	1	0				0																																																																																																							
0	0	0	0				0																																																																																																							
0	0	0	1				0																																																																																																							
(i) problem	(ii) optimal solution	(iii) optimal pseudo-solution																																																																																																												

Fig. 2. An example of a pseudo-solution

1.4 From pseudo-solutions to solutions

A dominating solution can be derived from a pseudo-solution as follows. For any product that appears in more than one set, remove all but one of its appearances. For example the pseudo-solution

$$(\{4\}, \{4\}, \{4, 5\}, \{\}, \{1, 2, 3\})$$

becomes

$$(\{4\}, \{\}, \{5\}, \{\}, \{1, 2, 3\})$$

if we delete all but the first appearance of each product. We now have the same number of products as sets, and can obtain a permutation by moving products without violating the ordering among sets. For example 5 can be moved one set to the left to obtain

$$(\{4\}, \{5\}, \{\}, \{\}, \{1, 2, 3\})$$

then 1 moved two sets to the left to obtain

$$(\{4\}, \{5\}, \{1\}, \{\}, \{2, 3\})$$

and finally 2 moved one set to the left to obtain the permutation

$$(\{4\}, \{5\}, \{1\}, \{2\}, \{3\})$$

It is clear that this transformation is correct and never increases Ω : removing a product cannot increase Ω , and it is always safe to move a product into a neighbouring empty set.

1.5 Increase in optimal solution density

The effect on optimal solution density can be spectacular. Consider a MOSP instance that we shall call A_N , represented by a $2 \times 2N$ matrix:

$$\left[\begin{array}{c|c} 00 \dots 0 & 1 \dots 11 \\ \hline 11 \dots 1 & 0 \dots 00 \end{array} \right]$$

The column permutation shown has 1 open stack in any column, and this is optimal. We may permute the left and right columns separately, and reverse all columns, to obtain other optimal permutations. Thus there are $2(N!)^2$ optimal permutations, and any other permutation such as

$$\left[\begin{array}{c|c} 0 \dots 0 \mathbf{101} & \dots 1 \\ \hline 1 \dots 1 \mathbf{010} & \dots 0 \end{array} \right]$$

has at least one column with two open stacks (because of the zeroes shown in **bold**). However, there are many more optimal pseudo-solutions. Considering only those cases in which all (0,1)-columns are in the N left (or right) columns and all (1,0)-columns are in the N right (or left) columns, there are $2(2^N - 1)^{2N}$ optimal pseudo-solutions. So the number of optimal solutions has been increased by a factor of at least

$$\frac{2(2^N - 1)^{2N}}{2(N!)^2} \approx \frac{2^{2N^2}}{(N!)^2} > \frac{2^{N^2}}{N^{2N}} = \frac{2^{N^2}}{2^{\log_2 N^{2N}}} = 2^{N^2 - 2N \log_2 N} \approx 2^{N^2}$$

Thus we have an exponential increase in solution density. At the other extreme, for some problems there will be exactly the same number of solutions in both models: in other words the reformulation introduces no pseudo-solutions. Consider a MOSP instance we shall call B_N , represented by the $2N \times N$ matrix

$$\left[\begin{array}{c} U_N \\ L_N \end{array} \right]$$

where U_N and L_N are upper and lower diagonal $N \times N$ matrices respectively:

$$U_N = \left[\begin{array}{c} 111 \dots 111 \\ 011 \dots 111 \\ 001 \dots 111 \\ \vdots \\ 000 \dots 001 \end{array} \right] \quad L_N = \left[\begin{array}{c} 100 \dots 000 \\ 110 \dots 000 \\ 111 \dots 000 \\ \vdots \\ 111 \dots 111 \end{array} \right]$$

Notice that every column has exactly $(N + 1)$ 1s and no 0 has a 1 to the left and the right, but in any other permutation this is no longer true. For example

in B_4 if we exchange the middle two columns:

$$\begin{bmatrix} 1111 \\ 0111 \\ 0011 \\ 0001 \\ 1000 \\ 1100 \\ 1110 \\ 1111 \end{bmatrix} \longrightarrow \begin{bmatrix} 1111 \\ 0111 \\ 0101 \\ 0001 \\ 1000 \\ 1010 \\ 1110 \\ 1111 \end{bmatrix}$$

then both have 6 open stacks instead of 5. So the matrix has two optimal solutions, each with $(N+1)$ open stacks: one is with U_N and L_N as shown, the other is obtained by reversing their columns. Notice also that each column has at least one 1 where the other has a 0 in the same row. Therefore no two columns can be placed in the same set without increasing the maximum number of open stacks, so there are no optimal pseudo-solutions.

1.6 An intermediate model

We shall refer to the original integer model as model 1, and the new one as model 3. We shall also consider an intermediate model 2 in which all sets in the pseudo-solution must be non-empty, which is obtained by replacing (1) by

$$\sum_k p_{jk} \geq 1 \quad \sum_j p_{jk} \geq 1 \quad (7)$$

Model 2 can also have exponentially more solutions than model 1. Considering only optimal pseudo-solutions to A_N in which product 1 appears in every set, there are $2(2^{N-1} - 1)^{2(N-1)}$ of them, which is still at least $O(2^{N^2})$ more than the number of model 1 solutions.

1.7 Model size

All three models have $O(R^2)$ p, l, r variables and $O(RC)$ o variables, giving a total of $O(R(R+C))$. There are $O(C)$ constraints (1) or (6) or (7) of size $O(R)$, $O(\Delta RC^2)$ constraints (2) of constant size where Δ is the matrix density,² $O(RC)$ constraints (3) of constant size, $O(RC)$ constraints (4) of constant size, and $O(C)$ constraints (5) of size $O(R)$, giving a total space complexity of $O(RC(1+C\Delta))$. The models are suitable for large problems with small Δ , in other words each customer orders a small number of products.

² Defined as the number of 1-entries divided by the total number of entries.

2 Experiments

We compare the three models using a new local search algorithm for (non-binary) integer programs. This is based on a recent SAT algorithm called VW2 [15], and uses the VW2 modified objective function that dynamically weights variables to improve search diversification. In all experiments below the noise parameter p is set to 0.05, the s parameter is set to 0.1, and the c parameter to 0.000001 (the algorithm will be described in future publications; see [15] for details on the parameters). These values were found to be robust across a range of problem instances. Results shown are the number of local moves (*flips*) required to find a solution. The differences in flip rates (number of flips per second) between the three models were negligible, as they have almost the same number of constraints.

For the A_N benchmark (with many optimal pseudo-solutions) the results are shown in Figure 3. The graph is a log-log plot so the straight lines show that search effort is polynomial in N . Models 2 and 3 are indistinguishable, but model 1 has a steeper gradient and therefore a higher polynomial degree. We also experimented with $N = 500$ and carefully tuned the search algorithm parameters, and the results were very similar: models 1 and 2 took tens of seconds to solve while model 3 took tens of minutes. A merely polynomial improvement may seem disappointing given the exponential increase in solution density. An explanation may be that most pseudo-solutions for this problem occur in the basins of attraction of real solutions. Nevertheless, the improvement is significant.

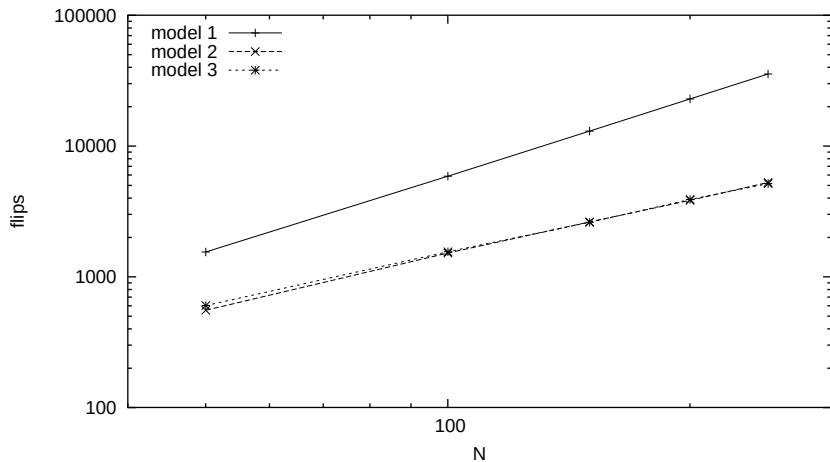


Fig. 3. Results on the A_N benchmarks

For the B_N benchmark (with no optimal pseudo-solutions) the results are shown in Figure 4 and are quite different. Models 1 and 2 are now similar, and have lower polynomial degree than model 3. An explanation may be that when

there are no pseudo-solutions the omitted constraints of model 3 are redundant because they exclude no solutions, and redundant constraints are known to aid local search on some problems [4, 13, 16].

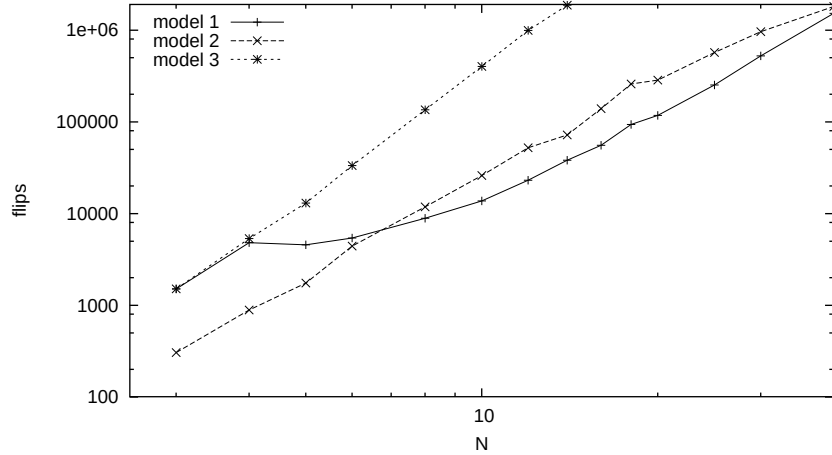


Fig. 4. Results on the B_N benchmarks

These results show that reformulation to increase solution density can be very beneficial for local search, but that when the solution density is unchanged it can be harmful. Care must therefore be taken to choose a robust model. On this problem model 2 is the most robust, behaving like the better of the other two models on each problem class.

To test the models further we experimented with the individual instances from the Constraint Modelling Challenge for which optimal solutions are known. We performed ten runs per instance and took the median number of flips taken to solve the problem to optimality.³ The results are shown in Table 1, table entries “—” denoting that the median number of flips was greater than 10^7 . The results are consistent with those above: model 2 again gives the most robust results, model 1 gives poor results, and model 3 sometimes gives the best and sometimes the worst results.

3 Related work

Dominated relaxed modelling is related to the *safe delay of constraints* [2, 3] in which certain constraints are not enforced until after the search has finished. Safe delay constraints have been applied to logical specifications of CSPs, and delaying them allows multiple assignments of values to CSP variables. A real

³ Optimal results were obtained by a complete algorithm, thanks to Nic Wilson.

problem	stacks	model 1	model 2	model 3
Miller19	13	2,453,224	227,425	157,637
GP1	45	6,785,315	2,022,414	4,940,263
GP2	40	—	1,386,156	8,902,904
GP3	40	—	2,829,814	—
GP4	30	8,729,306	4,901,462	—
NWRS1	3	2,475,203	29,472	17,102
NWRS2	4	670,150	122,263	29,004
NWRS3	7	2,819,739	113,894	42,625
NWRS4	7	231,640	167,744	50,389
NWRS5	12	693,360	60,100	79,034
NWRS6	12	332,528	260,164	51,750
NWRS7	10	—	577,905	559,957
NWRS8	16	4,159,016	1,458,881	432,312
SP1	9	1,076,624	222,584	832,262

Table 1. Results on selected challenge instances

solution can be obtained by taking any one of the multiple assignments to each variable. This is analogous to the removal of *at-most-one* clauses in a well-known SAT encoding of CSPs, which is known to improve local search performance [17, 19]. The constraints that we omit from our first model can be regarded as safe delay constraints for MOSP instead of SAT, and a more complex transformation is required to obtain solutions.

Local search on pseudo-solutions has some resemblance to Abstract Local Search (ALS) [6]. ALS solves combinatorial optimization problems by applying local search to a space of *abstract solutions* while ignoring details. For example, abstract solutions may be task prioritizations for a scheduling problem, and the details may be task start times. But there are differences between the approaches. Firstly, the objective function of the problem is not defined on abstract solutions, but it is defined on pseudo-solutions. Secondly, dominance plays a different role: any pseudo-solution must be dominated by a solution, whereas for any solution S there must exist an abstract solution that maps to a real solution that dominates S . Thirdly, pseudo-solutions are purely a modelling concept that may be exploited by any local search algorithm, whereas abstract solutions are used in a special search architecture. Pseudo-solutions require a polynomial-time algorithm for constructing solutions from them, which is used only once after search. Abstract solutions require a fast greedy algorithm to construct solutions from them, which may be invoked many times during search. Repairs to an abstract solution are derived by analysing conflicts in the solution derived from it.

Dominated relaxed modelling is also related to *supersymmetric modelling* [16, 18]. Any solution to a supersymmetric model is either a solution to the problem, or can be transformed to one by applying a symmetry transformation. Symmetric

solutions may be considered to dominate each other, so supersymmetry is a special case of dominated relaxed modelling.

4 Conclusion

Choosing a good model can be as important as choosing a good search algorithm. But modelling for local search and for complete search can require different methods, and relatively little work has been done on the former. Dominated relaxed modelling is a promising technique, though it may require a degree of ingenuity. The same is true of supersymmetric reformulations — perhaps because supersymmetry is the inverse of *symmetry breaking by reformulation*, which is known to be powerful but non-trivial to apply [10]. It is therefore unlikely that either technique can be automated. Nevertheless, they can have a significant effect on local search performance, and should form part of the constraint modeller’s toolbox.

Is dominated relaxed modelling suitable for use with complete search algorithms? It is the reverse of the usual application of dominance reasoning — to reduce the number of solutions and the size of the search space — which seems to imply that it will not work well with backtrackers. On the other hand, it is known that adding symmetry breaking constraints can have a positive or negative effect on the effort taken by a backtracker to find a solution, an effect that motivated the development of techniques such as SBDS [1, 11] and SBDD [7, 8]. If the effect of removing solutions by adding constraints is unpredictable then so (presumably) is the effect of creating new solutions by removing constraints. Thus we expect the effect of the technique on backtracker performance to be highly problem-dependent.

In future work we hope to find further applications of this technique. We also hope to analyse its effect on search space structure, as it is currently unproven that the performance benefits are caused by the increase in solution density.⁴

Acknowledgement

This material is based in part upon works supported by the Science Foundation Ireland under Grant No. 00/PI.1/C075.

References

1. R. Backofen, S. Will. Excluding Symmetries in Constraint-Based Search. *Fifth International Conference on Principles and Practice of Constraint Programming, Lecture Notes in Computer Science* vol. 1713, Springer-Verlag 1999, pp. 73–87.
2. M. Cadoli, T. Mancini. Automated Reformulation of Specifications by Safe Delay of Constraints. *Principles of Knowledge Representation and Reasoning: Proceedings of the Ninth International Conference*, AAAI Press, 2004, pp. 388–398.

⁴ Thanks to Ken Brown for pointing this out.

3. M. Cadoli, T. Mancini, F. Patrizi. SAT as an Effective Solving Technology for Constraint Problems. *Atti della Giornata di Lavoro: Analisi sperimentale e benchmark di algoritmi per l'Intelligenza Artificiale*, Dipartimento di Ingegneria, Università di Ferrara, Italy, 2005.
4. B. Cha, K. Iwama. Adding New Clauses for Faster Local Search. *Fourteenth National Conference on Artificial Intelligence*, American Association for Artificial Intelligence, 1996, pp. 332–337.
5. D. Clark, J. Frank, I. P. Gent, E. MacIntyre, N. Tomov, T. Walsh. Local Search and the Number of Solutions. *Second International Conference on Principles and Practices of Constraint Programming, Lecture Notes in Computer Science* vol. 1118, Springer, 1996, pp. 119–133.
6. J. M. Crawford, M. Dalal, J. P. Walser. Abstract Local Search. *AIPS'98 Workshop on Planning as Combinatorial Search*, Carnegie Mellon University, Pittsburgh, Pennsylvania, USA, 1998.
7. T. Fahle, S. Schamberger, M. Sellmann. Symmetry Breaking. *Seventh International Conference on Principles and Practice of Constraint Programming, Lecture Notes in Computer Science* vol. 2239, Springer, 2001, pp. 93–107.
8. F. Focacci, M. Milano. Global Cut Framework for Removing Symmetries. *Seventh International Conference on Principles and Practice of Constraint Programming, Lecture Notes in Computer Science* vol. 2239, Springer, 2001, pp. 77–92.
9. A. Fink, S. Voss. Applications of Modern Heuristic Search Methods to Pattern Sequencing Problems. *Computers and Operations Research* vol. 26, 1999, pp. 17–34.
10. I. P. Gent, J.-F. Puget. Symmetry Breaking in Constraint Programming. Tutorial, *Tenth International Conference on Principles and Practice of Constraint Programming*, Toronto, Canada, 2004.
11. I. P. Gent, B. M. Smith. Symmetry Breaking During Search in Constraint Programming. *Fourteenth European Conference on Artificial Intelligence*, Berlin, Germany, 2000, pp. 599–603.
12. Y. Hanatani, T. Horiyama, K. Iwama. Density Condensation of Boolean Formulas. *Sixth International Conference on the Theory and Applications of Satisfiability Testing, Lecture Notes in Computer Science* vol. 2919, Springer, 2003, pp. 69–77.
13. K. Kask, R. Dechter. GSAT and Local Consistency. *Fourteenth International Joint Conference on Artificial Intelligence*, Morgan Kaufmann, 1995, pp. 616–622.
14. A. Parkes. Clustering at the Phase Transition. *Fourteenth National Conference on Artificial Intelligence and Ninth Innovative Applications of Artificial Intelligence Conference* AAAI Press / MIT Press, 1997, pp. 340–345.
15. S. D. Prestwich. Random Walk With Continuously Smoothed Variable Weights. *Eighth International Conference on Theory and Applications of Satisfiability Testing, Lecture Notes in Computer Science* vol. 3569, Springer, 2005, pp. 203–215.
16. S. D. Prestwich. Negative Effects of Modeling Techniques on Search Performance. *Annals of Operations Research* vol. 118, Kluwer Academic Publishers, 2003, pp. 137–150.
17. S. D. Prestwich. Local Search on SAT-Encoded Colouring Problems. *Sixth International Conference on the Theory and Applications of Satisfiability Testing, Lecture Notes in Computer Science* vol. 2919, Springer, 2003, pp. 105–119.
18. S. D. Prestwich, A. Roli. Symmetry Breaking and Local Search Spaces. *Second International Conference on Integration of AI and OR Techniques in Constraint Programming for Combinatorial Optimization Problems, Lecture Notes in Computer Science* vol. 3524, Springer, 2005, pp. 273–287.

19. B. Selman, H. Levesque, D. Mitchell. A New Method for Solving Hard Satisfiability Problems. *Tenth National Conference on Artificial Intelligence*, MIT Press, 1992, pp. 440–446.
20. J. Singer, I. P. Gent, A. Smaill. Backbone Fragility and the Local Search Cost Peak. *Journal of Artificial Intelligence Research* vol. 12, 2000, pp. 235–270.
21. H. H. Yanasse. On a Pattern Sequencing Problem to Minimize the Maximum Number of Open Stacks. *European Journal of Operational Research* vol. 100, 1997, pp. 454–463.
22. M. Yokoo. Why Adding More Constraints Makes a Problem Easier for Hill-climbing Algorithms: Analyzing Landscapes of CSPs. *Third International Conference on Principles and Practice of Constraint Programming, Lecture Notes in Computer Science* vol. 1330, Springer-Verlag 1997, pp. 356–370.